

Vision Based Robot Navigation

Team Phoenix

1. Introduction and Goal

This main focus is developing an autonomous mobile robot capable of navigating a defined circuit arena, mapping its environment, and subsequently determining an optimized path for traversal. The system is designed to operate in two main phases: first, an exploration and mapping phase where the robot uses its sensors to build a representation of the arena, and second, a path planning phase where this map is processed to find an efficient route.

The primary objectives are:

- Autonomous navigation within the arena using camera-based guidance and obstacle avoidance.
- Simultaneous Localization and Mapping (SLAM), where the robot estimates its position and builds a 2D grid map of the environment, marking its own path and discovered free space.
- Post-processing of the generated map to identify a centerline path through the circuit.
- Future work includes pruning and smoothing this centerline to create an optimal, drivable path for the robot.

2. Initial Designs of the Robot

Mobility System

- **Drive Type:** Two-wheel differential drive with one caster wheel for support.
- **Motors:** Two **Webots Motor nodes** with **encoder support** enabled (`positionSensor` field).
- **Encoders:** Used for measuring wheel rotations, enabling feedback control (e.g., odometry, PID).

Frame

- **Custom Robot Model:** Built in Webots using `Solid`, `Shape`, and `Transform` nodes.
- The **top solid part of the robot** was designed externally and imported as a **custom STL file**.
- The rest of the body, including the drive base and caster mount, was modeled using built-in Webots shapes for ease of integration and performance.

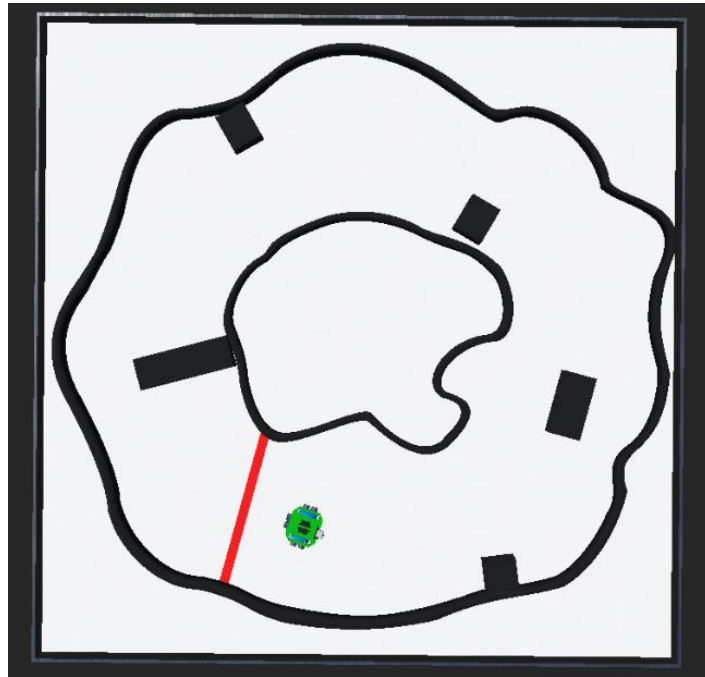


Figure 1 Custom built robot placed on the arena

Sensor Setup

Sharp IR Sensors

- **Simulated with:** DistanceSensor nodes in Webots.
- **Positioning:** Front and side-facing to detect obstacles and walls.
- **Use:** Trigger avoidance behavior and environmental awareness.

Camera

- **Webots Camera node** configured for RGB image input.
- **Use:** Visual tracking, navigation assistance, or future image processing pipelines.
- **Placement:** Front-mounted with slight downward tilt.

Inertial Unit

- **Webots InertialUnit node** used to track orientation (roll, pitch, yaw).
- **Use:** Directional feedback, turning accuracy, and heading estimation.

3. System Architecture and Modules

The project is modularized into several Python scripts, each handling specific functionalities:

- **config.py**: This script serves as the central configuration hub. It initializes the Webots Robot instance and its associated devices.
- **matrix_map.py**: This module contains the core logic for Simultaneous Localization and Mapping (SLAM).
- **movemap.py**: This module contains robot's movement control and navigation logic during the exploration phase.
- **map_json.py**: This utility module is used to save/load map from python using json files.
- **path_finder.py**: This script loads the map generated by the robot and implements an optimized path avoiding obstacles.
- **visualize.py**: This script is used for offline visualization of the maps saved as JSON files.
- **controller_th.py**: This is the main Webots controller script.

4. Arena Exploration

Initial Movement:

- Robot starts behind a red line and moves forward to cross it.

Red Line Detection:

- Continuously checks for red color in the center of the camera frame.
- Sets a flag when red is detected.
- Stops moving once red is no longer detected after being seen.

Image Strip Extraction:

- Extracts three horizontal strips from each camera frame:
 - middlestrip (central region)
 - bottomstrip (near the bottom of the frame)
 - blacktestStrip (between middle and bottom)

Pixel Classification:

- Converts each strip into arrays with values:
 - 1 for black

- 0 for white
- 5 for other colors

Track Detection:

- Identifies the longest run of white pixels (0) in each strip.
- Uses the midpoint of this run to estimate the center of the path.
- If only black or white pixels are found, handles edge cases accordingly.

Obstacle Avoidance:

- Uses the black test strip to check for black pixels at the edges (indices 20 and 280).
- Chooses the strip (middle or bottom) that is less likely to lead to a side collision.

Direction Decision:

- Calculates the error between the midpoint of the screen and the midpoint of the selected path.
- Uses a PID controller to adjust the robot's steering based on this error.

Motion Control:

- Continuously corrects the robot's direction using visual feedback to stay centered on the path and avoid obstacles.

To sum up, the robot processes camera images by analyzing specific horizontal strips to identify track edges or the centerline. This visual information is used to calculate how far the robot is from the ideal path. A PID controller then determines the necessary steering adjustments based on this error, allowing the robot to navigate smoothly along the track while avoiding obstacles.

5. Arena Mapping

Localization (Position Estimation)

Robots' initial position is known and passed to the controller initially. From there onwards robot maintains an estimate of its current 2D position and orientation in the arena. This is handled in `matrix_map.py`.

- **Orientation (Yaw):** The yaw angle is directly obtained from the Inertial Unit (IMU) sensor. This provides an absolute orientation relative to the world's coordinate system. In case of real world
- **Displacement:** Wheel encoders measure the rotation of each wheel. That angle is used with radius of wheels to calculate displacement from last updated position to find the displacement.

- **Pose Update:** The calculated linear distance is projected onto the X, Y axes using trigonometry. Then the current position is updated based on last position and displacement.

Map Representation

The environment is represented as a 2D grid map (MAP), which is a NumPy array.

- **Dimensions:** 100x100
- **Resolution:** 2cmx2cm per cel
- **Cell Values:**
 - 0 : Unexplored area.
 - 1 : Cells traversed by the robot.
 - 2 : Cells observed to be free by the side IR sensors.

Mapping Process

As the robot moves, it updates the MAP in each iteration:

- **Robot Path Marking:** marks a small block of cells (based on size of robot) around the robot's current calculated location.
- **Free Space Mapping (IR Sensors):**
 - The robot is equipped with two IR distance sensors, one on each side, with offset from the robot's center.
 - Free space is recognized by following procedure:
 1. The raw sensor value is read. Then it is converted to distance using the function provided in documentation of the product.
 2. The orientation of the sensor's beam is determined based on robots orientation.
 3. Then the cells addresses of matrix are calculated along this line of sight from the sensor's position up to the distance reading from sensor using trigonometry.
 4. Then all those cell addresses are updated as free spaces.

Map Storage

After the robot completes its mapping run, it save the current state of the MAP (NumPy array) into a JSON file. This file also stores metadata like map size and resolution.

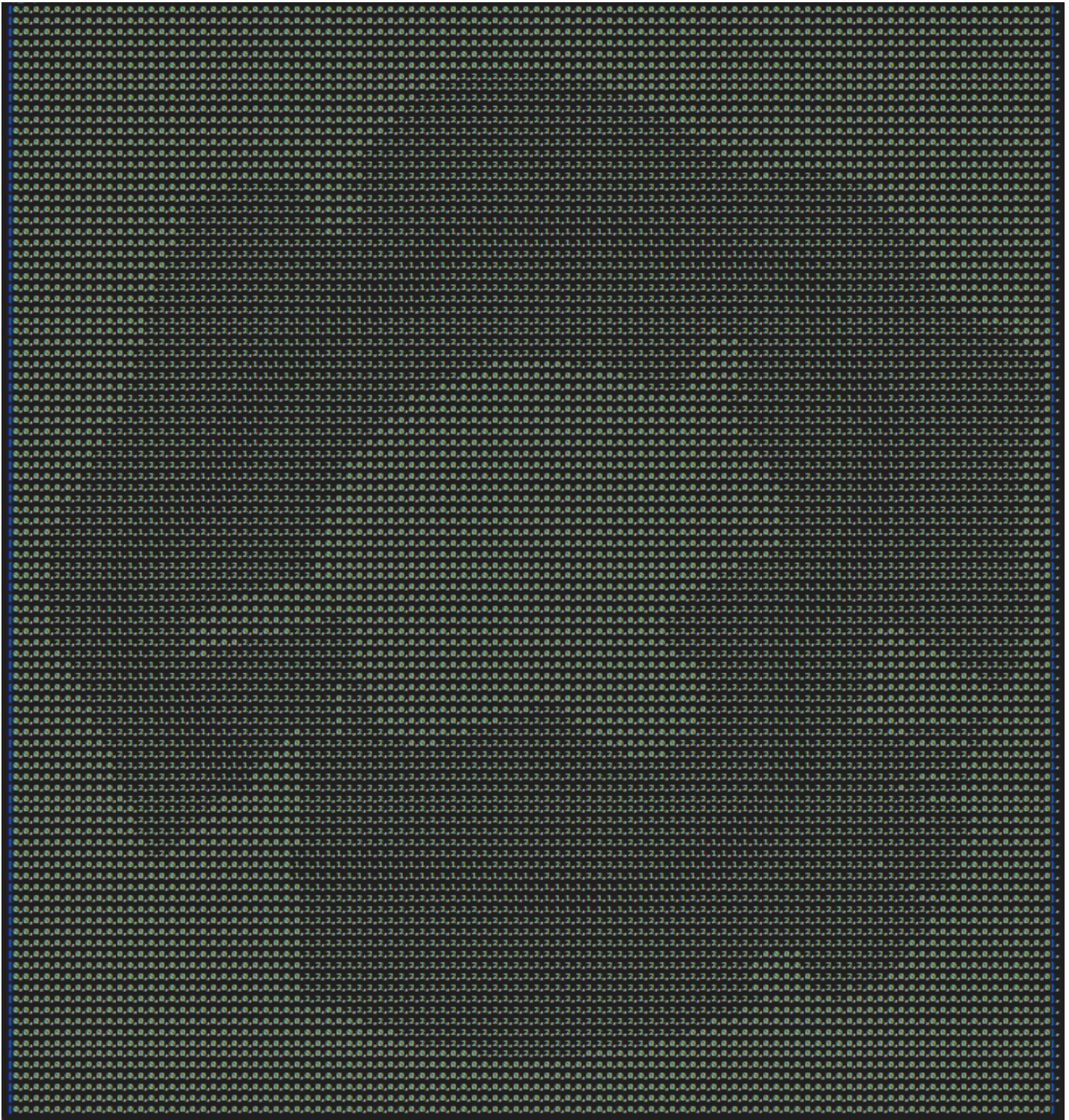


Figure 2- Sample Matrix of MAP built by robot after Initial run

6. Path Optimization

Based on the available initial map of arena an optimal path for movement is calculated.

Centerline Generation

The core of the initial path optimization is to find a centerline (or "spine") of the drivable area:

1. **Identify Traversable Area:** A binary map is created based on whether robot can move into each cell or not
2. **Skeletonization:** The skeletonize function from skimage.morphology is applied to this binary traversable map. This algorithm thins the traversable regions down to a one-pixel-wide representation, effectively creating the centerline.
3. **Path Ordering:**
 - All connected components within the generated skeleton are identified.
 - Then the largest connected component is considered, assuming represents the main track.
 - A Depth-First Search (DFS) algorithm is then used to traverse this largest component, starting from a specified preferred point.
 - This DFS traversal generates an ordered list of (row, column) waypoints representing the centerline.

Output of Path Finding

After the DFS run:

1. **Map with Ordered Path:** A new JSON file is created. This file contains the original map data but with the cells corresponding to the ordered centerline waypoints marked with sequentially increasing numbers (starting from 100).
2. **Ordered Waypoints List:** A separate JSON file (e.g., centerline_waypoints_ordered.json) is saved, containing only the list of ordered (row, col) waypoint coordinates, along with map resolution and size metadata. This list is intended for direct use by a path-following algorithm.

Then the robot can move without need of camera processing as it has defined a path. Or we can do the mapping for iterative rounds for better results.

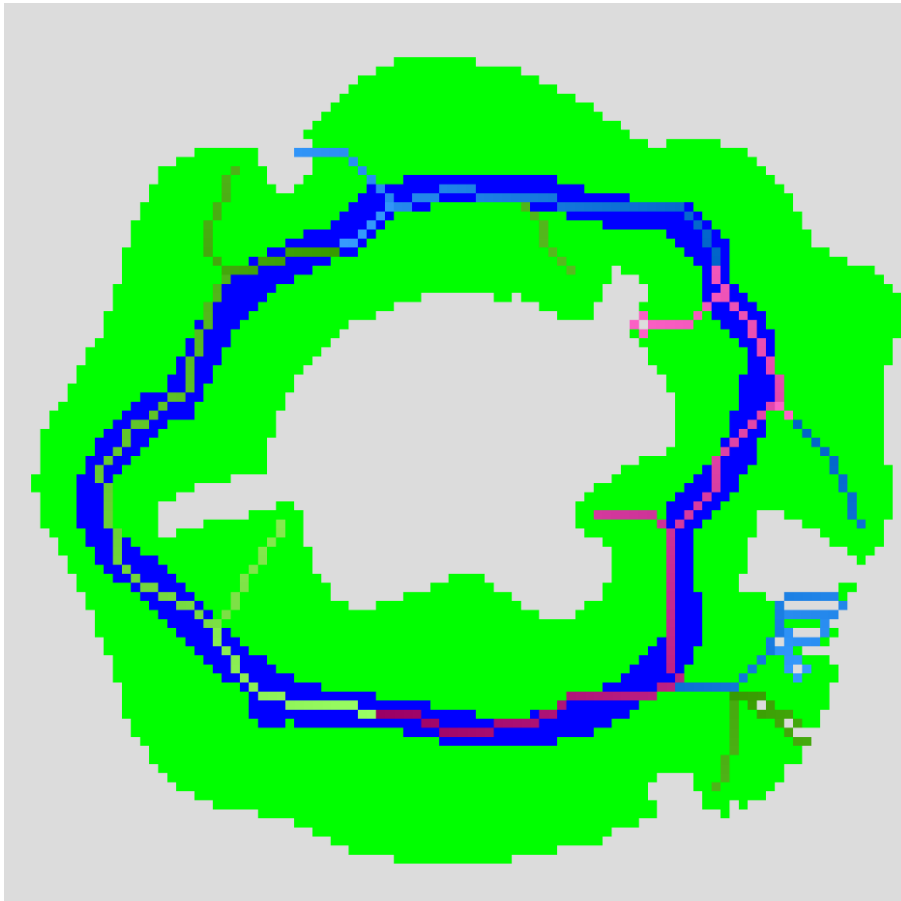


Figure 3 - After Generating centerline to map

7. Current Status and Next Steps

Current Status

- The robot successfully navigates the arena using camera-based control.
- It simultaneously estimates its position using IMU and encoders.
- It builds a 2D grid map, marking its own path and the free space detected by its side IR sensors.
- The populated map is saved to a JSON file.
- The path_finder.py script can load this map and generate a centerline (skeleton) of the traversable area.
- This centerline is ordered into a sequence of waypoints.
- The current centerline generation includes branches present in the skeleton of the largest free-space component.

Planned Next Steps

1. **Branch Pruning:** Implement an algorithm to prune the smaller, dead-end branches from the generated skeleton *before* the final path ordering. This will result in a cleaner centerline that more accurately represents the main circuit loop.
2. **Path Smoothing:** After obtaining a pruned and ordered list of centerline waypoints, apply path smoothing techniques. B-spline interpolation (e.g., using `scipy.interpolate.splprep` and `splev`) is likely a more suitable approach for generating a smooth, "elliptical curve" like path that the robot can follow efficiently with fewer abrupt turns. The smoothed path would then be discretized back into a sequence of waypoints for the robot.
3. **Path Following:** Develop a controller that enables the robot to accurately follow the generated (and eventually smoothed and pruned) sequence of waypoints.

Team Members:

- Shehan Perera
- Tharusha Dilshan
- Adheesha Kawshal
- Dhawala Rajakaruna
- Thisuka Kodithuwakku